University Admission Decision Engine

Problem statement

Evaluate applicants using subject eligibility, aggregate marks, course-specific thresholds, and tie-breaking criteria; generate a ranked list of eligible applicants.

Assumptions and rules

• Each candidate has name, Mathematics mark, Physics mark, Chemistry mark, and aggregate percentage.

• The course requires at least 60 in Mathematics, 50 in Physics, 50 in Chemistry, and 70 aggregate.

• Eligible applicants are ranked by aggregate percentage, then Mathematics mark, then Physics mark, all in descending order.

• Only lists, tuples, loops, conditions, and basic sorting logic are used.

Algorithm

1. Read candidate records and course thresholds.

2. For each candidate, check every subject minimum and the aggregate minimum.

3. Add eligible candidates to an eligible list with their ranking values.

4. Sort eligible candidates using nested loops and tuple comparisons, highest priority first.

5. Display the ranked eligible list and rejected candidates with reasons.

Pseudocode

```
START\nINPUT candidate records and course criteria\neligible = empty list\nFOR each candidate\n  CHECK subject minimums and
```

Flowchart

```
[Start] -> [Read candidate records and criteria]\n   |\n[Check subject minimums and aggregate]\n   |\n{All conditions satis
```

Python solution

The program uses beginner-level Python constructs from the assigned syllabus: variables, data types, operators, conditions, loops, strings, lists, tuples, and unpacking where appropriate.

```
candidates = [
    ("Asha", 85, 72, 70, 82.5),
    ("Bharat", 90, 65, 68, 78.0),
    ("Charan", 58, 80, 75, 76.0),
    ("Divya", 85, 72, 70, 82.5),
    ("Esha", 70, 55, 60, 71.0)
]
# Record: name, Maths, Physics, Chemistry, aggregate percentage

maths_minimum = 60
physics_minimum = 50
chemistry_minimum = 50
aggregate_minimum = 70

eligible = []
rejected = []

for candidate in candidates:
    name, maths, physics, chemistry, aggregate = candidate
    reasons = []

    if maths < maths_minimum:
        reasons.append("Maths below minimum")
    if physics < physics_minimum:
        reasons.append("Physics below minimum")
    if chemistry < chemistry_minimum:
        reasons.append("Chemistry below minimum")
    if aggregate < aggregate_minimum:
```

```
        reasons.append("Aggregate below minimum")

    if len(reasons) == 0:
        eligible.append(candidate)
    else:
        rejected.append((name, reasons))

# Selection sort: aggregate, then Maths, then Physics, descending
for i in range(len(eligible)):
    best = i
    for j in range(i + 1, len(eligible)):
        if eligible[j][4] > eligible[best][4]:
            best = j
        elif eligible[j][4] == eligible[best][4]:
            if eligible[j][1] > eligible[best][1]:
                best = j
            elif eligible[j][1] == eligible[best][1]:
                if eligible[j][2] > eligible[best][2]:
                    best = j
    temporary = eligible[i]
    eligible[i] = eligible[best]
    eligible[best] = temporary

print("RANKED ELIGIBLE CANDIDATES")
rank = 1
for candidate in eligible:
    print(rank, candidate[0], "Aggregate:", candidate[4],
          "Maths:", candidate[1], "Physics:", candidate[2])
    rank = rank + 1

print("REJECTED CANDIDATES")
for item in rejected:
    print(item[0], "-", ", ".join(item[1]))
```

Sample output

```
RANKED ELIGIBLE CANDIDATES\n1 Asha Aggregate: 82.5 Maths: 85 Physics: 72\n2 Divya Aggregate: 82.5 Maths: 85 Physics: 72\n3 
```

Explanation / observations

Asha and Divya remain tied after all specified tie-breakers; their relative order is the original input order. A further tie-breaker can be added if the university requires one.

Conclusion

This solution models the stated case using a clear sequence of input, processing, decisions, repetition, and output. The assumptions and sample data can be changed to test other cases.